

Аннотация

Выпускная квалификационная работа посвящена разработке веб-приложения мультимодального поиска и анализа данных на основе RAG-архитектуры и LLM-модуля. Актуальность исследования обусловлена стремительным ростом объемов разнородной информации, представленной в виде текстов, изображений, аудио- и видеоданных, для эффективной обработки которых требуется применение интеллектуальных методов поиска.

В работе рассмотрены методы мультимодального поиска и обработки данных, проведен анализ существующих решений на основе генеративных языковых моделей и архитектуры Retrieval-Augmented Generation. Выполнено проектирование архитектуры веб-приложения, разработана структура RAG-пайплайна и механизм обработки мультимодальных данных с использованием существующих моделей. Реализовано программное обеспечение, включающее модуль формирования векторных представлений, поиск релевантного контекста и генерацию ответов на основе LLM/VLM. Проведено тестирование системы и оценка ее работоспособности.

Практическая ценность работы заключается в возможности использования разработанного веб-приложения для интеллектуального поиска и анализа мультимодальной информации в образовательных, исследовательских и корпоративных информационных системах.

Общий объем работы составляет 52 страницы основного текста, 5 страниц приложений.

Содержание

Содержание.....	3
Перечень сокращений и обозначений.....	4
Введение.....	5
1. Анализ методов и технологий мультимодального поиска и обработки данных с применением генеративных языковых моделей.....	8
1.1 Методы и технологии мультимодального поиска на основе генеративных моделей.....	8
1.2 Архитектура Retrieval-Augmented Generation и принципы ее работы..	10
1.3 Анализ современных исследований и решений в области мультимодального поиска на основе RAG.....	12
2. Проектирование веб-приложения мультимодального поиска и анализа данных на основе RAG-архитектуры и LLM-модуля.....	15
2.1. Общая характеристика разрабатываемого веб-приложения.....	15
2.2. Пользовательские сценарии и требования к системе.....	19
2.3. Проектирование архитектуры веб-приложения.....	24
2.4. Проектирование структуры данных и RAG-пайплайна.....	27
3. Программная реализация и тестирование веб-приложения мультимодального поиска и анализа данных на основе RAG-архитектуры и LLM-модуля.....	32
3.1. Выбор технологий и средств разработки.....	32
3.2. Реализация веб-приложения.....	39
3.3. Тестирование и анализ результатов работы системы.....	44
Заключение.....	48
Список использованных источников.....	50
Приложения.....	56

Перечень сокращений и обозначений

1. ВКР — выпускная квалификационная работа
2. ПО — программное обеспечение
3. ОС — операционная система
4. СУБД — система управления базами данных
5. RAG — Retrieval-Augmented Generation, генерация с дополненной выборкой
6. LLM — Large Language Model, большая языковая модель
7. VLM — Vision Language Models, визуально-языковая модель
8. Zero-shot — это метод поиска информации (текста, изображений), при котором модель находит релевантные данные без предварительного обучения на конкретных примерах или задачах
9. VQA-инструменты — это инструменты для Visual Question Answering: на вход подаются изображение + текстовый вопрос, на выходе модель возвращает текстовый ответ.
10. text-to-image — это задача поиска изображений по текстовому запросу.
11. few-shot-подход — это подход, при котором модель решает новую задачу, имея только несколько примеров.

Введение

Развитие технологий обработки данных и искусственного интеллекта приводит к появлению новых подходов к поиску и анализу информации. Современные программные системы ориентированы на работу с большими массивами данных, которые отличаются по структуре, способу хранения и семантическому содержанию. Важной задачей становится создание решений, способных объединять текстовые и визуальные данные в едином поисковом пространстве, обеспечивая высокую точность результатов при сохранении приемлемого времени отклика. Одними из основных направлений развития в данной области являются методы машинного обучения и обработки естественного языка, основанные на больших языковых моделях (Large Language Models [1]) или визуально-языковых моделях (Vision Language Models [2]).

Актуальность исследования обусловлена стремительным ростом объёмов разнородной информации, обрабатываемой современными информационными системами. Значительная часть таких данных представлена в виде мультимодальных источников: текстовых документов, изображений, видео, табличных структур, данных с датчиков и технической документации. Традиционные методы полнотекстового поиска оказываются недостаточно эффективными в условиях, когда требуется обнаружение семантических связей между фрагментами данных, принадлежащих к различным типам модальностей. В связи с этим возникает необходимость разработки технологий, обеспечивающих интеллектуальный поиск и анализ мультимодальной информации.

Объектом исследования являются системы мультимодального поиска и анализа данных.

Предмет исследования — методы и технологии реализации мультимодального поиска на основе архитектуры Retrieval-Augmented Generation [3] и генеративных языковых моделей, включая формирование эмбеддингов, хранение векторных представлений и генерацию ответов.

Цель данной научно-исследовательской работы заключается в разработке веб-приложения поиска и анализа мультимодальных данных на основе архитектуры генерации с дополненной выборкой и модулей генеративных языковых моделей.

Для достижения поставленной цели необходимо **решить следующие задачи**:

1. Провести анализ предметной области мультимодального поиска и анализа данных, выявить проблему и сформулировать требования к решению.
2. Разработать интеллектуальную модель мультимодального поиска и анализа данных на основе больших языковых моделей и RAG-архитектуры, включающую механизмы интеграции текстовых и визуальных данных, семантического поиска, генерации обоснованных ответов, а также обеспечения релевантности, достоверности и согласованности результатов.
3. Спроектировать и реализовать веб-приложение в виде информационной системы для поиска и анализа мультимодальных данных с интеграцией визуально-языковой модели.
4. Оценить эффективность разработанного решения на тестовых примерах по заранее заданным метрикам и провести анализ результатов.

Проблема исследования заключается в том, что генеративные архитектуры позволяют осуществлять осмысленные ответы на разнородные запросы пользователя, однако они лишены механизма доступа к внешним

источникам знаний. В условиях отсутствия контекстной информации наблюдается повышенная вероятность генерации недостоверных ответов — галлюцинаций.

Теоретическая значимость исследования заключается в систематизации фундаментальных методов обработки мультимодальных данных, сравнительном анализе технологий векторного поиска и формализации комплекса требований к веб-приложениям данного класса.

Структура научно-исследовательской работы соответствует требованиям методических рекомендаций и включает аннотацию, оглавление, перечень сокращений и обозначений, введение, три основных раздела, заключение, список использованных источников и приложения.

В первой главе рассматриваются теоретические и методологические основы мультимодального поиска и анализа данных, архитектуры RAG и генеративные языковые модели, а также анализируются существующие подходы и решения.

Во второй главе выполняется проектирование веб-приложения мультимодального поиска и анализа данных: определяются пользовательские сценарии, формулируются требования, разрабатываются архитектура системы и структура RAG-пайплайна.

Третья глава посвящена программной реализации разработанного решения и его тестированию, включая описание процесса разработки, механизма поиска и генерации ответов, а также анализ работоспособности системы.

1. Анализ методов и технологий мультимодального поиска и обработки данных с применением генеративных языковых моделей

1.1 Методы и технологии мультимодального поиска на основе генеративных моделей

Мультимодальный поиск ориентирован на обработку и сопоставление данных различных типов, включая текст, изображения и видеоматериалы, в рамках единого поискового пространства. В отличие от классического полнотекстового поиска, такой подход предполагает поиск не только по ключевым словам, но и по семантическому сходству объектов, представленных в различных модальностях. [4] Технологической основой подобных систем выступают совместные представления текста и изображений в общем векторном пространстве, что позволяет выполнять поиск по близости эмбедингов и учитывать смысловые связи между объектами.

Одним из ключевых методов реализации мультимодального поиска является формирование эмбедингов — векторных представлений документов, изображений и иных объектов. Такие представления строятся с использованием нейросетевых моделей и затем сохраняются во внешних хранилищах для последующего поиска ближайших соседей. [5, 6] В работах по хранению векторных представлений подчеркивается, что данный подход позволяет свести задачу поиска к вычислению расстояния между векторами, а также делает возможной эффективную обработку мультимодальных документов. При этом с ростом объемов данных возникает задача оптимизации хранения эмбедингов, включая методы сжатия и

кластеризации, позволяющие снизить требования к памяти и повысить вычислительную эффективность системы. [4]

Современные мультимодальные системы все чаще строятся на базе визуально-языковых моделей и генеративных языковых моделей. Предобученные модели такого класса формируют совместное латентное пространство для текста и изображений, что делает возможным как zero-shot поиск, так и более сложные сценарии составленного поиска, в которых пользователь комбинирует эталонное изображение и текстовое уточнение. [4, 7] При этом исследования показывают, что стандартные VLM хорошо работают на частотных и простых запросах, но испытывают трудности при композиционных запросах, редких случаях и примерах вне распределения, что требует дополнительных механизмов оптимизации поиска и улучшения интерпретации запроса. [4]

Важным направлением развития является использование генеративных моделей не только как средства формирования ответа, но и как компонента управления поисковым процессом. В современных архитектурах LLM могут интерпретировать естественно-языковой запрос, учитывать контекст предыдущих взаимодействий, преобразовывать пользовательский ввод к формату, понятному модели поиска, и координировать вызов различных инструментов поиска. [7] Такой подход делает интерфейс более естественным для пользователя и приближает мультимодальный поиск к формату интеллектуального ассистента.

Таким образом, к основным технологиям мультимодального поиска на основе генеративных моделей относятся: эмбединги, векторные базы данных, визуально-языковые модели, а также использование LLM для интерпретации запросов, управления поисковыми инструментами и генерации итогового ответа. [4, 5, 7] Совместное использование этих компонентов позволяет строить более гибкие и интеллектуальные системы.

1.2 Архитектура Retrieval-Augmented Generation и принципы ее работы

Архитектура Retrieval-Augmented Generation представляет собой подход, в котором генеративная языковая модель дополняется механизмом поиска релевантной информации во внешних источниках данных. В отличие от изолированной LLM, формирующей ответ только на основе внутренних параметров, RAG-система извлекает релевантный контекст из базы знаний и использует его при генерации ответа. [6, 8] Подобная архитектура активно рассматривается как в исследованиях по корпоративным ассистентам, так и в системах мультимодального поиска.

Классическая реализация RAG включает несколько последовательных этапов. Сначала документы проходят этап предварительной обработки: выполняются очистка, разбиение на фрагменты и формирование эмбеддингов. Затем векторные представления сохраняются во внешнем хранилище, пригодном для семантического поиска. При поступлении пользовательского запроса система строит его векторное представление, выполняет поиск релевантных фрагментов и передает найденный контекст в языковую модель, которая формирует итоговый ответ. [6, 8] Таким образом, базовый цикл работы RAG можно представить как последовательность «индексация — извлечение — генерация». [6]

В современных мультимодальных системах RAG-архитектура может быть дополнена механизмами, позволяющими обрабатывать не только текстовые, но и визуальные входы. В статье о мультимодальном поиске на основе LLM предложена архитектура, в которой используются поиск только по изображениям, мультимодальный поиск и VQA-инструменты, а LLM выступает в роли оркестратора, координирующего выбор нужного инструмента и учитывающего историю взаимодействия с пользователем. [7]

Такой подход позволяет реализовать conversational search, при котором пользователь может начинать с изображения, затем уточнять запрос текстом и получать релевантные результаты с учетом контекста предыдущих шагов. [7]

С архитектурной точки зрения, в состав RAG-системы входят несколько ключевых компонентов: модуль формирования эмбеддингов, векторная база данных, механизм поиска, генеративная модель и пользовательский интерфейс. Для мультимодальных задач дополнительно используются визуально-языковые модели, энкодеры изображений и компоненты анализа изображений. [6, 7]

Практическая эффективность RAG-архитектуры напрямую зависит от качества индексируемых данных, выбора модели эмбеддингов, структуры хранилища и корректности этапа поиска. В прикладных исследованиях по корпоративным ассистентам подчеркивается, что слабым местом таких систем остается не только сама LLM, но и необходимость постоянного обновления базы знаний, нормализации документов, защиты конфиденциальных данных и контроля качества результатов. [6, 8] Следовательно, RAG нужно рассматривать не как одиночную модель, а как комплексную архитектуру, в которой поиск и генерация тесно связаны с вопросами инфраструктуры и управления данными.

Таким образом, RAG-архитектура является одной из наиболее перспективных основ для построения систем интеллектуального поиска и анализа данных. Ее значение особенно велико в задачах мультимодального поиска, где необходимо не только находить релевантные объекты в разнородных источниках, но и формировать осмысленный ответ на естественном языке с опорой на внешний контекст. [7, 8]

1.3 Анализ современных исследований и решений в области мультимодального поиска на основе RAG

Современные исследования в области мультимодального поиска развиваются в направлении объединения методов семантического поиска, генеративных языковых моделей и внешних хранилищ знаний. В зарубежных работах мультимодальный поиск рассматривается как система, способная сопоставлять текстовые и визуальные представления объектов в едином семантическом пространстве, а также использовать языковую модель для интерпретации запросов и управления поисковыми инструментами. В этом контексте генеративные модели перестают быть только средством формирования ответа и начинают выступать как оркестратор, связывающий пользовательский запрос, механизм поиска и итоговую генерацию. [7]

Одним из важных направлений зарубежных исследований является разговорный мультимодальный поиск, при котором пользователь взаимодействует с системой в естественно-языковой форме и может последовательно уточнять запрос. В таких архитектурах используются отдельные инструменты для поиска только по изображениям, мультимодальный поиск и визуального анализа изображений, а LLM координирует их работу, учитывая историю предыдущих запросов. [7]

Заметное место в современных исследованиях занимает составленный поиск и text-to-image поиск, где запрос формируется не только как текст, но и как сочетание изображения-образца и текстового уточнения. Такие задачи особенно характерны для мультимодального поиска, поскольку требуют не просто сравнения объектов, а учета изменения признаков, заданных пользователем. Исследования показывают, что сильные визуально-языковые модели могут испытывать трудности при работе со сложными композиционными запросами, редкими объектами, логическими

ограничениями и сценариями вне распределения. Для повышения качества поиска предлагается few-shot-подход, при котором текстовый запрос дополняется небольшим числом эталонных положительных и отрицательных примеров. [4]

С технической точки зрения мультимодальный поиск опирается на формирование эмбедингов и использование векторных хранилищ. Документы, изображения и другие объекты преобразуются в векторные представления, после чего поиск выполняется по принципу семантической близости. При увеличении масштаба коллекции данных существенное значение приобретают методы индексирования, хранения и сжатия эмбедингов. В отечественных исследованиях показано, что применение кластеризации и хранения дельт относительно центроидов позволяет уменьшить издержки хранения при приемлемом уровне точности, что важно для масштабируемых мультимодальных поисковых систем. [5]

В отечественных исследованиях по RAG-архитектуре основное внимание уделяется поисковым системам и корпоративным ассистентам, которые работают с внутренними базами знаний, документацией и организационными регламентами. Подчеркивается, что использование RAG позволяет существенно снизить галлюцинации ответов языковых моделей за счет обращения к внешним данным. В одном из рассмотренных исследований показано, что использование генерации с дополненной выборкой приводит к заметному росту точности ответов языковых моделей по сравнению с режимом работы без внешней выборки. Это подтверждает практическую значимость RAG для задач, где критичны достоверность, актуальность и привязка ответа к источнику данных. [8]

Анализ зарубежных и отечественных решений позволяет выделить несколько устойчивых направлений развития мультимодального поиска на основе RAG. Во-первых, происходит переход от статического поиска к

диалоговому взаимодействию, в котором запрос может уточняться по ходу общения с системой. Во-вторых, растет значимость использования векторных хранилищ и методов эффективного хранения эмбедингов как инфраструктурной основы для этапа поиска. В-третьих, особое внимание уделяется качеству обработки сложных композиционных запросов и устойчивости системы к нестандартным сценариям. [4, 5, 7]

Несмотря на достигнутые результаты, мультимодальный поиск на основе RAG по-прежнему включает в себя ряд проблем. К ним относятся сложность формирования единого представления для объектов различных модальностей, снижение качества поиска при сложных композиционных запросах, необходимость постоянного обновления базы знаний, а также зависимость итогового качества ответа от корректности этапа поиска. Кроме того, при построении прикладных систем необходимо учитывать масштабируемость векторного хранилища, время отклика и согласованность между компонентом поиска и генеративной моделью. [4, 5, 7]

Таким образом, современные исследования показывают, что наиболее перспективным направлением развития мультимодального поиска является интеграция генеративных языковых моделей, векторного поиска и RAG-архитектуры. Такой подход позволяет объединить возможности семантического поиска, работы с разнородными данными и формирования осмысленных ответов на естественном языке, что делает его особенно актуальным для разработки интеллектуальных веб-приложений поиска и анализа мультимодальной информации. [5, 6, 7]

2. Проектирование веб-приложения мультимодального поиска и анализа данных на основе RAG-архитектуры и LLM-модуля

2.1. Общая характеристика разрабатываемого веб-приложения

Предлагаемое программное решение представляет собой веб-систему интеллектуального мультимодального поиска и анализа данных, основанную на архитектуре RAG. Система предназначена для автоматизированного извлечения информации из разнородных источников, преобразования данных в векторную форму, хранения их во внешнем хранилище и формирования смысловых ответов с использованием генеративной языковой модели. Основная идея разработки заключается в объединении методов семантического векторного поиска и механизмов генерации естественно-языковых ответов, что позволяет повысить точность, достоверность и релевантность результатов, а также снизить вероятность формирования недостоверных выводов.

Архитектура решения включает несколько функциональных модулей, взаимодействующих между собой по принципу модульного или микросервисного построения. Серверная часть обеспечивает обработку пользовательских запросов, координацию этапов поиска и генерации, а также взаимодействие с внешними хранилищами и вычислительными компонентами. Для реализации логики RAG используется специализированный программный слой, обеспечивающий построение цепочек обработки данных, формирование эмбедингов, поиск релевантного контекста и передачу его в генеративную модель.

Хранение векторных представлений организовано во внешней векторной базе данных, поддерживающей поиск ближайших соседей,

фильтрацию по метаданным и масштабируемую обработку коллекций документов. Это обеспечивает возможность быстрого извлечения релевантных фрагментов данных при работе с большими массивами мультимодальной информации.

Помимо специализированного векторного хранилища, в составе системы предусматривается использование реляционной базы данных общего назначения, предназначенной для хранения служебной и прикладной информации. В такой базе данных будут размещаться учетные записи пользователей, сведения о загруженных документах, метаданные объектов, история запросов и результатов поиска, данные аутентификации и иные сущности, не требующие представления в виде эмбедингов. Совместное использование реляционного и векторного хранилищ позволяет разделить контуры хранения семантических представлений и операционных данных, что повышает структурированность системы, упрощает сопровождение и обеспечивает более гибкое управление информацией.

Особое внимание в проектируемой системе уделяется масштабируемости и устойчивости при увеличении объема данных и числа пользовательских запросов. Для этого предусматривается использование средств распределённой обработки, позволяющих распараллеливать вычислительные операции, связанные с векторизацией, индексированием и обработкой запросов. Развёртывание компонентов системы ориентировано на изолированную и отказоустойчивую модель эксплуатации, обеспечивающую гибкость обновления и сопровождения отдельных модулей.

Клиентская часть приложения предназначена для выстраивания взаимодействия пользователя с системой, формирования запросов и генерации ответов на основе модели. Интерфейс должен обеспечивать удобную работу с мультимодальными данными, поддержку интерактивных

сценариев, отображение результатов и наглядное представление сгенерированных ответов с источниками.

Дополнительные технологические инструменты в составе решения обеспечивают управление зависимостями, автоматизацию сборки, контроль качества исходного кода и подготовку технической документации. Это повышает поддерживаемость программного продукта а и упрощает дальнейшее развитие проекта.

Важной частью предлагаемого решения является наличие средств мониторинга, позволяющих отслеживать производительность отдельных компонентов, анализировать нагрузку и своевременно выявлять потенциальные сбои. Такой подход обеспечивает предсказуемость поведения системы в эксплуатационной среде и повышает её отказоустойчивость.

К основным задачам разрабатываемой системы относятся:

- извлечение информации из разнородных источников с последующей нормализацией данных;
- преобразование данных в векторные представления и их размещение во внешнем хранилище;
- выполнение поиска по векторным коллекциям с учётом семантического сходства;
- генерация релевантных ответов на основе найденных фрагментов данных;
- снижение вероятности возникновения галлюцинаций генеративной модели за счёт применения RAG-подхода;
- обеспечение высокой производительности, масштабируемости и доступности программной системы;
- мониторинг технических параметров и поддержка качества разработки.

В процессе разработки и эксплуатации системы предусматривается использование подходов непрерывной интеграции и непрерывной поставки, направленных на автоматизацию проверки, сборки, тестирования и развертывания программных компонентов. Применение таких подходов позволяет повысить стабильность программного решения, ускорить внедрение изменений и минимизировать риски, связанные с обновлением отдельных модулей системы.

Таким образом, разрабатываемое решение представляет собой архитектурную платформу для построения систем интеллектуального поиска, сочетающую методы обработки естественного языка, векторный поиск, генеративные модели и средства промышленной эксплуатации. Такая структура обеспечивает расширяемость, сопровождаемость и возможность интеграции с внешними информационными системами.

2.2. Пользовательские сценарии и требования к системе

Ключевые пользовательские сценарии работы разрабатываемой системы включают загрузку мультимодальных данных, формирование поискового запроса, извлечение релевантных фрагментов информации из внешнего хранилища, генерацию ответа на основе найденного контекста, а также просмотр и анализ результатов взаимодействия. Пользователь может работать с системой как в режиме поиска по уже загруженным данным, так и в режиме пополнения информационной базы новыми документами и объектами. Указанные сценарии определяют состав функциональных возможностей системы и служат основой для формулирования требований к ее архитектуре, производительности и удобству использования.

Разрабатываемая система ориентирована не только на общий сценарий мультимодального поиска, но и на решение ряда конкретных прикладных задач. В качестве типовых сценариев практического применения можно выделить поиск по технической документации, анализ учебных материалов и работу с мультимодальными исследовательскими данными. Выделение таких сценариев позволяет конкретизировать предметную область применения системы и уточнить требования к ее функциональности.

Первый сценарий связан с поиском по технической документации. В данном случае пользователь работает с инструкциями, регламентами, проектной документацией, техническими описаниями, схемами, таблицами и иллюстрациями. Система должна обеспечивать загрузку таких документов, их индексацию, извлечение текстового и визуального содержимого, а также поиск релевантных фрагментов по естественно-языковому запросу. Практическая ценность данного сценария заключается в сокращении времени поиска нужной информации в больших массивах технических материалов и повышении точности доступа к нужным данным.

Второй сценарий связан с анализом учебных материалов. В этом случае система может использоваться для работы с лекциями, методическими пособиями, презентациями, лабораторными работами и иными образовательными ресурсами. Пользователь получает возможность задавать вопросы по содержанию загруженных материалов, находить нужные фрагменты, сопоставлять текстовую и визуальную информацию и получать сгенерированный ответ с опорой на конкретные источники. Такой сценарий демонстрирует применимость системы в образовательной среде и показывает, что мультимодальный поиск может использоваться как инструмент интеллектуальной поддержки учебного процесса.

Третий сценарий связан с обработкой исследовательских и аналитических материалов, включающих текстовые документы, изображения, таблицы и иные данные. В данном случае система может применяться для поиска по отчетам, аналитическим материалам, экспериментальным описаниям и сопутствующим визуальным объектам. Пользователь получает возможность выполнять поиск не только по тексту, но и по связанному мультимодальному контексту, что особенно важно в задачах, где существенная часть информации представлена в графическом или смешанном виде.

Таким образом, практическое применение разрабатываемой системы может быть конкретизировано через несколько предметно-ориентированных сценариев. Это позволяет рассматривать веб-приложение не как абстрактную универсальную платформу, а как инструмент, пригодный для решения задач поиска и анализа данных в технической, образовательной и исследовательской среде. На основе указанных сценариев далее формируются функциональные и нефункциональные требования к системе.

Функциональные требования задают перечень операций, которые разрабатываемая система мультимодального поиска должна выполнять при штатных условиях эксплуатации. [9]

Таблица 2.2.1 — Функциональные требования

Требование	Описание
Загрузка данных	Система обеспечивает возможность загрузки файлов различных типов, включая текстовые и графические форматы.
Извлечение информации	Система выполняет автоматическое извлечение текстовых фрагментов и сопутствующих метаданных из загруженных документов.
Формирование векторных представлений	Система преобразует извлечённые данные в числовые векторные представления для последующей обработки.
Размещение данных во внешнем хранилище	Система сохраняет сформированные векторные представления и связанные метаданные во внешнем хранилище.
Семантический поиск	Система обеспечивает поиск наиболее релевантных записей на основе вычисления схожести векторных представлений.
Контекстная генерация ответов	Система формирует естественно-языковой ответ на основании данных, найденных в результате поиска.
Управление доступом	Система предоставляет механизм аутентификации пользователей для ограничения несанкционированного доступа.

Ведение истории взаимодействия	Система регистрирует запросы пользователей и сохраняет результаты поиска для последующего анализа.
--------------------------------	--

Совокупность зафиксированных функциональных требований охватывает полный цикл обработки данных в системе: от загрузки и извлечения информации до семантического поиска, генерации ответов и журналирования действий пользователей. Тем самым задается минимально необходимый набор функций, обеспечивающих реализацию мультимодального поиска на основе RAG-подхода и поддержку ключевых пользовательских сценариев. Данные требования также служат основой для последующей разработки плана тестирования, позволяя формализовать критерии проверки корректности работы отдельных модулей и системы в целом.

Нефункциональные требования задают совокупность качественных характеристик разрабатываемой системы и определяют ограничения, в рамках которых должны выполняться её функциональные возможности. К таким характеристикам относятся показатели производительности, масштабируемости, надёжности, безопасности, удобства использования и соблюдения нормативных требований. В отличие от функциональных требований, описывающих, *что* именно делает система, нефункциональные требования фиксируют, *как* она должна это делать с точки зрения качества, ресурсов и условий эксплуатации. Они оказывают прямое влияние на выбор архитектурных решений и технологии реализации.[9]

Таблица 2.2.2 — Нефункциональные требования

Категория	Требование
Масштабируемость	Архитектура системы должна поддерживать возможность

	горизонтального увеличения вычислительных ресурсов.
Безопасность	Система должна реализовывать проверку подлинности пользователей и защиту данных в процессе взаимодействия.
Удобство использования (UX)	Пользовательский интерфейс должен корректно отображаться на устройствах с различными разрешениями экранов и обеспечивать интуитивное взаимодействие с системой.
Конфиденциальность	Обработка данных должна соответствовать действующему законодательству в области защиты персональной информации.

Заданные нефункциональные требования формируют условия для проектирования и эксплуатации системы мультимодального поиска и анализа данных, определяя допустимые значения ключевых метрик и ограничений. На их основе могут разрабатываться сценарии нагрузочного и отказоустойчивого тестирования, а также критерии оценки готовности программного решения к выпуску как продуктивной версии. Также, нефункциональные требования служат инструментом согласования ожиданий между разработчиками, заказчиком и потенциальными пользователями, обеспечивая единое понимание требуемого уровня качества системы.

2.3. Проектирование архитектуры веб-приложения

Архитектура программного обеспечения — совокупность важнейших решений об организации программной системы.

Архитектура включает:

1. выбор структурных элементов и их интерфейсов, с помощью которых составлена система, а также их поведения в рамках сотрудничества структурных элементов;
2. соединение выбранных элементов структуры и их поведения во всё более крупные системы;
3. архитектурный стиль, который направляет всю организацию — все элементы, их интерфейсы, их сотрудничество и их соединение.

Для предлагаемого решения используется архитектура, которая делит клиентский интерфейс, серверную обработку запросов, модуль поиска и модуль генерации ответов. Такой подход повышает гибкость разработки, облегчает масштабирование отдельных компонентов и обеспечивает независимость модулей при внесении изменений в исходный код системы [10].

Уровень клиентского интерфейса обеспечивает передачу запросов на сервер, обработку ответов и управление состоянием пользовательского интерфейса. Коммуникация осуществляется по протоколу HTTP или WebSocket, что позволяет реализовать как синхронные, так и интерактивные сценарии работы.

Серверный уровень объединяет функции маршрутизации запросов, выполнения бизнес-логики, обращения к внешним сервисам и координации потоков обработки данных. На данном уровне выполняется валидация входных данных, обработка пользовательских запросов, организация

семантического поиска и последующая генерация ответа. В рамках RAG-архитектуры серверный слой обеспечивает преобразование входных данных в формат, пригодный для семантического сопоставления, выполнение векторного поиска по хранилищу данных, отбор релевантных фрагментов контекста и передачу найденной информации генеративной модели. Таким образом, серверный уровень выступает связующим звеном между клиентской частью, хранилищами данных и внешними вычислительными компонентами, обеспечивая унифицированный интерфейс взаимодействия и независимость пользовательского интерфейса от внутренних механизмов обработки.

В составе серверного уровня также реализуются функции работы с данными, включая размещение метаданных и векторных представлений, а также выполнение операций добавления, удаления, обновления и выборки записей. Такое объединение делает серверную часть центральным узлом системы, в котором сосредоточена логика обработки запроса: от приема пользовательского ввода до генерации итогового ответа с опорой на внешний контекст.

Отдельным модулем архитектуры является модуль фоновой и распределённой обработки данных. Его назначение заключается в выполнении ресурсоемких и асинхронных операций. К таким операциям относятся индексация загружаемых файлов, извлечение и предварительная обработка содержимого, формирование эмбеддингов, обновление векторных коллекций, а также повторная обработка данных при переиндексации. Отделение данного модуля от основного серверного узла позволяет разгрузить его, сократить время отклика пользовательских запросов и обеспечить параллельное выполнение вычислительных задач.

Использование отдельного модуля фоновой обработки особенно важно при работе с большими массивами данных и при одновременном поступлении большого числа запросов. За счет распределения задач между

независимыми процессами или узлами обеспечивается масштабируемость системы и повышается ее устойчивость к росту нагрузки. Такой подход позволяет выполнять обработку файлов, построение векторных представлений и обновление индексов параллельно, без блокировки основного контура взаимодействия с пользователем. В результате архитектура сохраняет приемлемое время отклика и обеспечивает стабильную работу системы даже при увеличении объема данных и интенсивности эксплуатации.

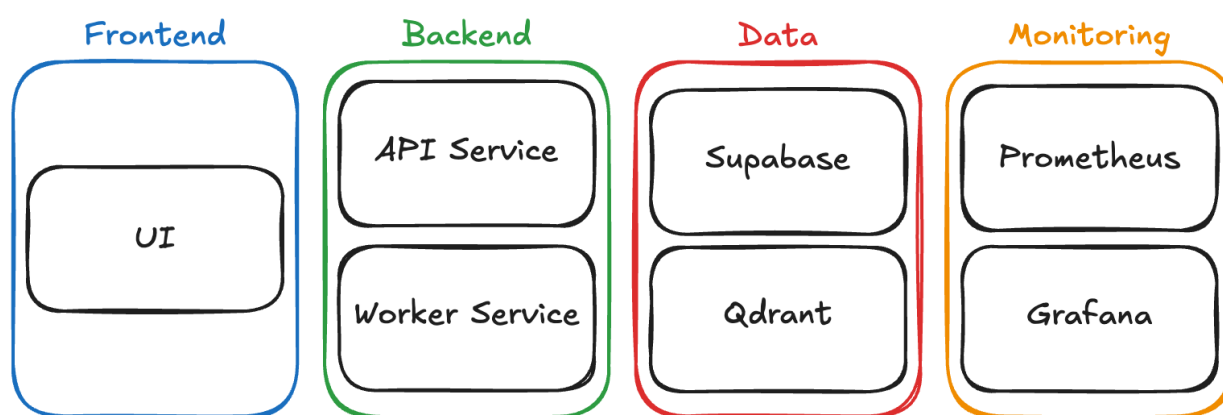


Рисунок 2.3.1 — Схема компонентов веб-приложения

Представленная схема иллюстрирует компонентную архитектуру разрабатываемого веб-приложения мультимодального поиска. В структуре системы выделены четыре основные группы компонентов: клиентская часть, серверная часть, слой хранения данных и подсистема мониторинга.

2.4. Проектирование структуры данных и RAG-пайплайна

Проектирование структуры данных и RAG-пайплайна является ключевым этапом разработки веб-приложения мультимодального поиска, поскольку именно на этом уровне определяется, каким образом будут организованы хранение файлов, метаданных, векторных представлений и история пользовательского взаимодействия, а также каким образом будет происходить обработка документов и формирование ответа на запрос пользователя. В рамках разрабатываемой системы используется комбинированный подход к хранению информации, при котором исходные файлы, операционные данные и векторные представления размещаются в различных контурах хранения в зависимости от их назначения.

Исходные пользовательские файлы сохраняются в файловом хранилище, предназначенном для размещения загруженных документов и мультимодальных объектов до их полной обработки. Такой подход позволяет отделить физическое хранение исходного контента от логики индексирования и последующего поиска. Метаданные файлов, сведения о пользователях, информация о папках, заданиях на индексацию и истории запросов размещаются в реляционном хранилище. Использование отдельной базы для операционных данных позволяет поддерживать целостность записей, формировать структуру пользовательской базы знаний и обеспечивать управление жизненным циклом загруженных объектов. Векторные представления документов и их фрагментов, напротив, размещаются во внешнем векторном хранилище, оптимизированном для семантического поиска по эмбедингам и извлечения релевантного контекста.

Логическая структура данных системы включает несколько основных сущностей. Сущность файлов хранит сведения о загруженных документах, включая идентификатор файла, имя, тип, размер и путь хранения. Сущность

папок используется для организации иерархической структуры пользовательской базы знаний и логической группировки файлов. Сущность заданий на индексацию предназначена для управления асинхронной обработкой загруженных объектов и фиксирует статус выполнения, количество попыток и результат обработки. Для задач, не завершившихся успешно после повторных попыток, используется отдельный контур хранения ошибок индексации. История запросов содержит пользовательские вопросы, сформированные ответы и сведения об использованных источниках, что позволяет сохранять результаты взаимодействия и использовать их для последующего анализа. Такая структура обеспечивает разделение данных по их роли в системе и упрощает сопровождение архитектуры.

Проектирование RAG-пайплайна в разрабатываемой системе включает два взаимосвязанных контура: пайплайн загрузки и индексации файла и пайплайн обработки пользовательского запроса. Первый контур отвечает за подготовку данных к последующему поиску, а второй — за извлечение релевантной информации и генерацию ответа на ее основе.

Пайплайн загрузки и индексации начинается с действия пользователя, который загружает файл в систему. После этого выполняется сохранение файла и его метаданных, а затем создается задача на индексацию. Обработка таких задач выполняется не в основном пользовательском контуре, а в фоновом режиме, что позволяет не блокировать интерфейс и не увеличивать время отклика системы. На этапе фоновой обработки осуществляется извлечение содержимого файла, его разбор и подготовка к дальнейшей векторизации. В зависимости от типа входного объекта система может извлекать текстовые фрагменты, описания изображений, элементы таблиц или иные данные, пригодные для дальнейшей семантической обработки. После этого формируются эмбединги, которые затем записываются во внешнее векторное хранилище. Завершающим этапом является обновление

статуса индексации, отражающее успешность или ошибку обработки. Общая последовательность данных этапов представлена на рисунке 2.4.1.

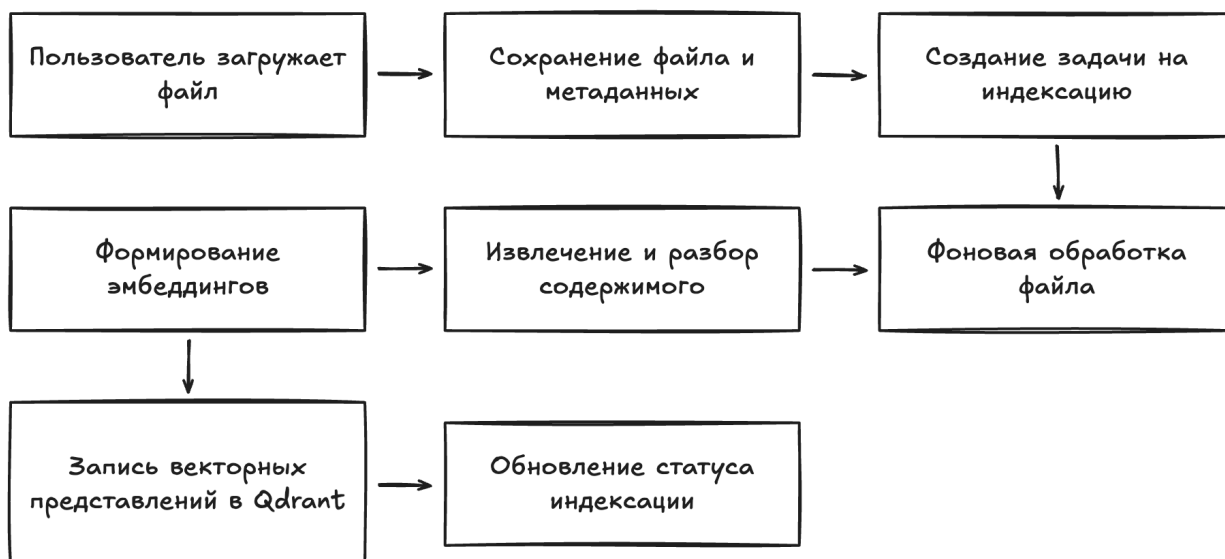


Рисунок 2.4.1 — Пайплайн загрузки и индексации файла в системе

Представленный на рисунке 2.4.1 пайплайн показывает, что индексирование реализуется как отдельный асинхронный процесс, включающий сохранение объекта, постановку задачи в очередь, обработку содержимого, построение эмбедингов и запись результатов в векторную базу. Такая организация позволяет обеспечить устойчивость системы при работе с большими файлами, уменьшить нагрузку на основной серверный контур и поддерживать масштабируемую обработку данных.

Второй контур представляет собой мультимодальный RAG-пайплайн, используемый при обработке пользовательского запроса. Его логика основана на том, что исходные данные предварительно проходят этап разбора, в ходе которого из них извлекаются текстовые, табличные и визуальные элементы. После извлечения содержимого отдельные компоненты могут быть дополнительно обработаны: текст используется напрямую, изображения, видео и таблицы могут интерпретироваться и преобразовываться в текстовое, а затем в векторное представление. Сформированные представления

записываются в векторное хранилище, которое в дальнейшем используется как внешний источник контекста для RAG.

При поступлении пользовательского вопроса система преобразует запрос в форму, пригодную для семантического сопоставления, и выполняет поиск релевантных данных по векторной базе. На этапе поиска из векторного хранилища извлекаются наиболее близкие по смыслу фрагменты, связанные с текстом, изображениями или иными элементами документа. Полученный контекст передается в генеративную модель, которая формирует итоговый ответ на естественном языке. Таким образом, генерация осуществляется не изолированно, а с опорой на внешний источник знаний, что позволяет уменьшить вероятность галлюцинаций и повысить достоверность ответа. Данный процесс отражен на рисунке 2.4.2.

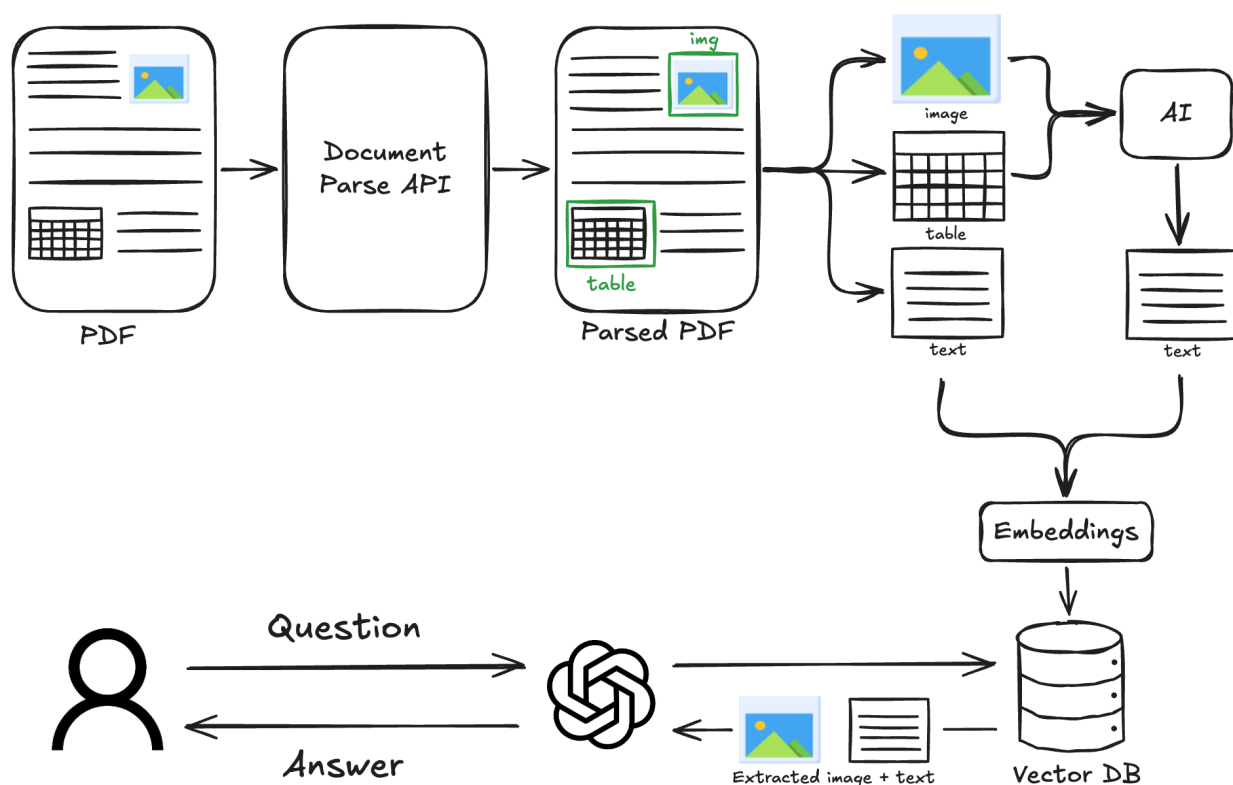


Рисунок 2.4.2 — Пайплайн мультимодальной RAG-архитектуры

Особенность проектируемого мультимодального RAG-пайплайна заключается в том, что контекстом могут служить не только текстовые фрагменты, но и результаты обработки изображений, таблиц и прочих данных. Это позволяет применять систему к разнородным источникам информации, существенно расширяя возможности поиска по сравнению с классическими текстовыми RAG-сценариями. При этом единое векторное представление контента обеспечивает возможность семантического сопоставления данных различных типов в рамках одного поискового механизма.

Спроектированная структура данных и RAG-пайплайн дают четкое понимание и разделение контуров хранения, обработки и поиска информации. Файловое хранилище отвечает за "оригиналы" (объекты), реляционная база отвечает за оперативные сущности и пользовательские данные, а векторное хранилище отвечает за семантический поиск по эмбедингам. Асинхронная индексация позволяет обрабатывать большие объемы данных без ухудшения пользовательского опыта, а мультимодальный RAG-пайплайн помогает вытащить релевантный контекст и сгенерировать ответ, используя внешние источники. Такой подход формирует основу для построения масштабируемой системы интеллектуального мультимодального поиска и анализа данных.

3. Программная реализация и тестирование веб-приложения мультимодального поиска и анализа данных на основе RAG-архитектуры и LLM-модуля

3.1. Выбор технологий и средств разработки

Технологический стек программного решения должен обеспечивать целостность системы мультимодального поиска, включая процессы векторизации данных, хранение векторных представлений, управление вычислительными ресурсами и взаимодействие с генеративными моделями. Важным критерием выбора технологий является не только функциональность отдельных инструментов, но и степень их интегрируемости, расширяемости и совместимости в рамках единой архитектуры.

В качестве языка программирования серверной части целесообразно использовать Python - язык программирования с развитой экосистемой библиотек машинного обучения. Это позволит получить доступ к широчайшему спектру фреймворков, предназначенных для работы с эмбединговыми моделями, генеративными архитектурами и векторными хранилищами, а также минимизировать затраты на интеграцию отдельных компонентов системы [11].

Ключевым компонентом взаимодействия с сервером выступает фреймворк FastAPI [12], направленный на создание высокопроизводительных REST API [13] и WebSocket-интерфейсов [14]. В отличие от традиционных решений, таких как Flask [15] или Django REST Framework [16], FastAPI поддерживает асинхронную обработку запросов, что существенно снижает задержку и повышает скорость реакции системы при высокой нагрузке. Данный фреймворк также предоставляет встроенные средства

автоматической документации API (Swagger/OpenAPI), что упрощает интеграцию клиентских приложений [17].

Для реализации механизмов RAG применяется инструмент LangChain, обеспечивающий формирование последовательностей обработки данных, включающих этапы векторизации, поиска релевантных документов и генерацию естественно-языкового ответа [18]. Выбор данной технологии обусловлен наличием встроенной поддержки генеративных моделей, модулей взаимодействия с внешними провайдерами (OpenAI, HuggingFace [19; 20]), а также реализацией шаблонов построения сложных RAG-конвейеров, включая поиск, пост-обработку результатов.

LangChain является предпочтительным решением для систем мультимодального поиска благодаря следующим преимуществам [21]:

- унифицированный интерфейс взаимодействия с различными источниками данных;
- возможность использования нескольких стратегий поиска (dense search, hybrid search, multi-query);
- поддержка сохранения контекста и цепочек рассуждений (chain-of-thought).

Векторные базы данных являются одним из ключевых элементов архитектуры мультимодального поиска. Они предназначены для хранения многомерных представлений данных и обеспечения поиска ближайших соседей в высокоразмерных пространствах. Наиболее известными системами являются Qdrant и pgvector [22; 23].

На основании проведённого анализа в таблице 3.1.1 наиболее предпочтительным решением для реализации системы мультимодального поиска является Qdrant.

В качестве реляционной базы данных используется Supabase. В составе системы используется для размещения прикладных и служебных данных, не представляемых в виде эмбедингов. К таким данным относятся учетные записи пользователей, сведения о загруженных файлах и папках, задания на индексацию, история запросов и иные метаданные, обеспечивающие работу пользовательской базы знаний. В качестве основы такого хранилища целесообразно использование платформы, построенной на PostgreSQL, поскольку она сочетает возможности надежной реляционной базы данных с готовыми средствами аутентификации и управления доступом.

Дополнительным преимуществом является простота развертывания и интеграции с серверной частью приложения, что позволяет сократить затраты на разработку инфраструктурных компонентов и сосредоточиться на реализации прикладной логики системы. Использование подобного решения обеспечивает централизованное хранение операционных данных, поддержку механизмов авторизации «из коробки» и удобство сопровождения веб-приложения.

Для обеспечения мобильности и изолированности компонентов системы применяется контейнеризация с использованием технологии Docker [28], которая позволяет независимо управлять сервисами Qdrant и клиентской частью. Контейнерный подход обладает следующими преимуществами:

- снижает вероятность конфликтов зависимостей;
- обеспечивает воспроизводимость окружений;
- упрощает интеграцию с CI/CD-конвейерами.

Для управления развертыванием микросервисов используется система Kubernetes, предоставляющая механизмы автоматического масштабирования, балансировки нагрузки и мониторинга состояния контейнеров [29]. Применение Kubernetes позволяет реализовать отказоустойчивую

конфигурацию, при которой сбой одного узла не препятствует функционированию системы в целом. Характеристики технологий серверной части, контейнеризации и оркестрации представлены в таблицах 3.1.2 и 3.1.3 соответственно.

Клиентская часть системы играет ключевую роль в обеспечении взаимодействия пользователя с веб-приложением, визуализации результатов мультимодального поиска и формировании запросов к серверной части. От состояния пользовательского интерфейса зависит не только удобство эксплуатации системы, но и общая эффективность обработки информации, поскольку корректная и интуитивная форма представления данных способствует снижению когнитивной нагрузки и ускоряет анализ полученных результатов. Для реализации интерфейса была выбрана связка современных технологий, обеспечивающих высокую производительность, гибкость и расширяемость приложения.

Важной основой клиентской части выступает язык TypeScript, позволяющий применять статическую типизацию для контроля взаимодействия между компонентами приложения [30]. В отличие от традиционного JavaScript [31], TypeScript имеет механизмы типобезопасности, что снижает вероятность логических ошибок, упрощает отладку и повышает надёжность клиентского кода при работе с асинхронными запросами к серверу. Это особенно важно в системах мультимодального поиска, где обрабатываемые данные могут иметь сложную структуру, включающую текстовые фрагменты, метаданные и векторные представления.

Фреймворк Nuxt 4 [32], построенный на основе Vue 3 [33], обеспечивает высокий отклик интерфейса и поддерживает гибкие сценарии маршрутизации и серверного рендеринга (SSR [34]). Применение Nuxt позволяет оптимизировать начальную загрузку страницы, повышает скорость

отображения результатов и улучшает показатели Core Web Vitals [35], что критично для приложений, работающих с большим количеством динамического контента. Компонентный подход Vue 3 упрощает проектирование интерфейсов, позволяя реализовывать визуальные элементы как автономные блоки с чётко определённой функциональностью.

Для обеспечения единообразного и адаптивного оформления элементов пользовательского интерфейса используется Tailwind CSS, представляющий собой библиотеку для быстрой стилизации [36]. Tailwind позволяет формировать дизайн на основе множества стилизующих классов, что минимизирует использование каскадных таблиц стилей вручную и ускоряет разработку интерфейса. При этом обеспечивается адаптивность интерфейса, что особенно важно для отображения результатов мультимодального поиска на устройствах с различными разрешениями экрана.

Инструментом сборки клиентской части является Vite, обеспечивающий высокую скорость разработки и низкое время отклика при горячей перезагрузке модулей [37]. По сравнению с классическими инструментами, такими как Webpack [38], Vite существенно ускоряет процесс компиляции и обеспечивает эффективную работу с зависимостями, что снижает время разработки и повышает качество итогового приложения. Использование Vite также позволяет легко интегрировать дополнительные инструменты, такие как ESLint [39], Prettier [40] и средства End-to-End тестирования.

Таким образом, представленные технологии в совокупности обеспечивают надёжность и высокую производительность. Это позволяет реализовать удобный для пользователя интерфейс при сохранении высокой эффективности разработки и сопровождения системы. Характеристики технологий клиентской части представлены в таблице 3.1.4.

Разработка системы мультимодального поиска на основе RAG-архитектуры требует использования не только основных инструментов обработки данных и серверного взаимодействия, но и дополнительных технологий, обеспечивающих контроль качества кода, оптимизацию процессов разработки, мониторинг состояния системы и автоматизацию документирования.

Для эффективного управления зависимостями в среде разработки используется инструмент UV [41], представляющий собой высокопроизводительный менеджер пакетов и окружений для Python. В отличие от стандартных решений, таких как `pip` [42], Poetry [43] или Conda [50], UV обеспечивает низкое время установки пакетов, поддержку изоляции окружений, ускоренные операции кеширования и воспроизводимость версий зависимостей. Это становится критически важным при работе с большим количеством библиотек для машинного обучения, векторной обработки данных и интеграции с LangChain. Сравнительный анализ технологий управления пакетами представлен в таблице 3.1.5.

Для контролем над эксплуатационными метриками системы используются инструменты мониторинга Prometheus [44] и визуализации данных Grafana [45]. В условиях распределённых вычислений и интенсивной работы векторного хранилища наличие аналитических механизмов для отслеживания состояния сервисов является обязательным элементом инфраструктуры.

- Prometheus выполняет сбор и хранение метрик в формате временных рядов (time-series), фиксируя состояние отдельных компонентов системы: задержку запросов FastAPI, скорость векторного поиска в Qdrant и использование оперативной памяти.

- Grafana предоставляет интерфейс визуализации и построения интерактивных панелей, что позволяет оперативно выявлять аномалии, отслеживать состояние RAG-цепочек и прогнозировать нагрузку при масштабировании.

Использование Prometheus и Grafana в связке позволяет осуществлять полноценный мониторинг производительности и своевременное обнаружение ошибок, повышая надёжность и отказоустойчивость системы.

С целью предотвращения ошибок при разработке используется инструмент pre-commit [46], обеспечивающий автоматическое выполнение проверок перед фиксацией изменений в систему контроля версий.

Для структурирования технической документации и автоматизации её подготовки используется генератор статических сайтов mkdocs [47], основанный на языке разметки Markdown [48].

Преимущества mkdocs включают:

- генерацию документации в виде веб-сайта;
- работу с множеством расширений (схемы Mermaid [49], таблицы, подсветка кода);
- совместимость с CI/CD для автоматического обновления документации при изменении исходного кода.

Документирование снижает порог входа для новых участников проекта и способствует качественной поддержке системы на всех этапах её жизненного цикла.

Использование подобных технологий снижает операционные риски при эксплуатации программного комплекса и упрощает его дальнейшую эволюцию без существенных затрат на поддержку. Характеристика дополнительных технологий представлена в таблице 3.1.6.

3.2. Реализация веб-приложения

Программная реализация разработанного веб-приложения мультимодального поиска и анализа данных выполнена в виде многокомпонентной информационной системы, которая состоит из пользовательского интерфейса, серверного API-модуля, подсистемы фоновой обработки и индексации, файлового, реляционного и векторного хранилища, а также модуля генерации ответов. Такая организация позволяет разделить пользовательский контур взаимодействия, контур асинхронной обработки данных и контур Retrieval-Augmented Generation, что повышает устойчивость, расширяемость и сопровождаемость программного решения.

Пользовательский интерфейс реализован как одностраничное приложение, где можно выполнять поиск в виде диалога, загружать файлы, работать с базой знаний и просматривать истории запросов. Интерфейс состоит из трех основных блоков: центральная область диалога, панель базы знаний и история запросов, что позволяет пользователю работать с документами и запросами в рамках единого рабочего пространства. Центральная форма ввода поддерживает передачу текстового запроса, выбор модели, прикрепление файла и ограничение области поиска по папкам или отдельным файлам. В результате пользователь получает не только сгенерированный ответ, но и список источников, использованных на этапе retrieval, что повышает прозрачность работы RAG-системы.

В приложении реализованы два пользовательских сценария загрузки данных: разовое прикрепление файла к запросу и постоянная загрузка документов в пользовательскую базу знаний. Во втором случае пользователь может управлять папками и файлами, повторно индексировать документы, удалять их и использовать для ограничения области поиска. Для авторизованных пользователей предусмотрено сохранение истории запросов и ответов, что обеспечивает воспроизводимость ранее выполненных поисковых сценариев. Система поддерживает гостевой и авторизованный

режимы работы: в гостевом режиме доступны разовые запросы и временные вложения, а в авторизованном — персональная база знаний, история и расширенные операции с документами.

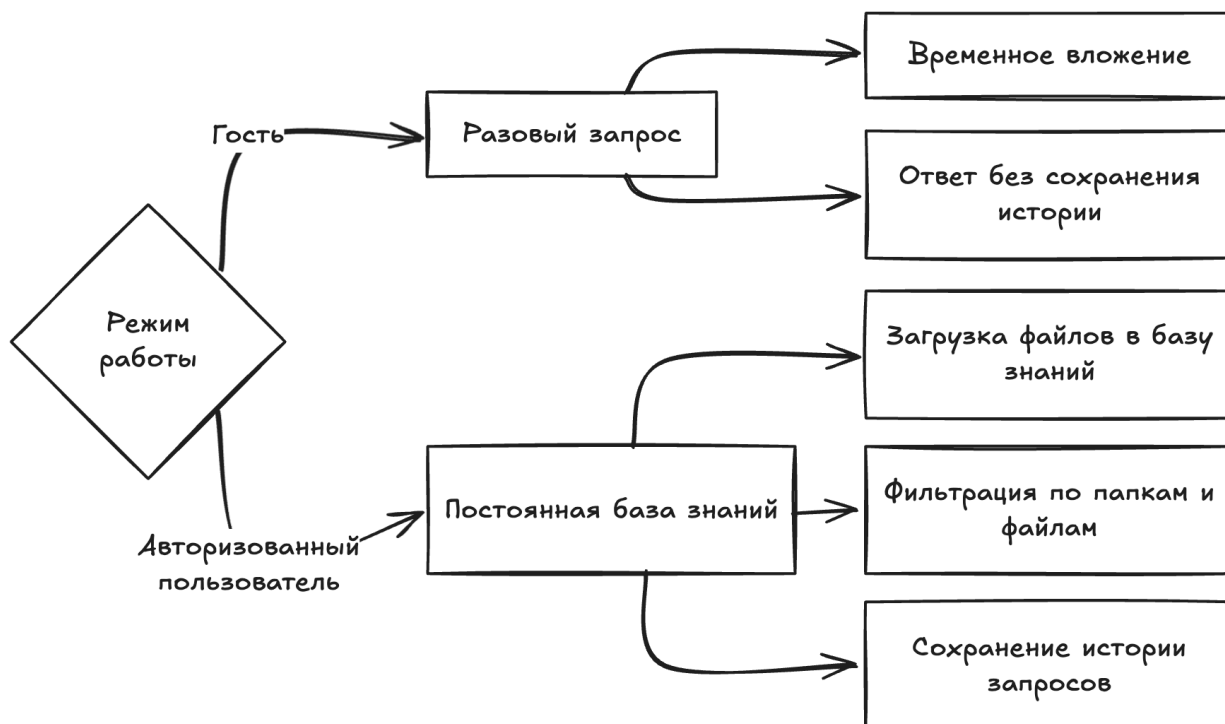


Рисунок 3.2.1 — Режимы работы пользователя

Серверная часть реализована на языке Python с использованием FastAPI и включает API-уровень, бизнес-сервисы, ядро обработки embeddings, RAG и LLM, слой взаимодействия с хранилищами и модуль мониторинга. Такое разделение позволяет изолировать обработку HTTP-запросов от бизнес-логики и инфраструктурных операций. Серверный уровень отвечает за валидацию входных данных, маршрутизацию запросов, работу с базой знаний, организацию индексации, выполнение retrieval и передачу контекста генеративной модели.

Для хранения данных используются несколько специализированных контуров. Исходные файлы сохраняются в файловом хранилище, метаданные файлов, папок, заданий индексации и истории запросов размещаются в

реляционной базе данных, а эмбединги документов и мультимодальных объектов — во векторном хранилище. Такое разделение позволяет разграничить операционные и поисковые данные, а также повысить эффективность обработки пользовательских запросов. Основные сущности хранения приведены в таблице 3.2.1.

Таблица 3.2.1 — Основные сущности хранения данных

Сущность	Назначение	Основное содержимое
kb_files	Хранение метаданных загруженных файлов	Идентификатор файла, пользователь, папка, имя, MIME-тип, размер, путь хранения, хэш
kb_folders	Организация структуры пользовательской базы знаний	Идентификатор папки, имя, родительская папка
ingest_jobs	Очередь задач индексации	Идентификатор задачи, статус обработки, параметры запуска
ingest_jobs_dlq	Хранение ошибочных задач индексации	Неуспешно обработанные задачи после нескольких попыток
query_history	История пользовательских запросов и ответов	Запрос, ответ, retrieved-документы, дата создания
Qdrant collections	Хранение векторных представлений	Текстовые, визуальные и видео-эмбединги с метаданными

После загрузки файла создаётся задача на индексацию, которая передаётся в модуль фоновой обработки. В ходе индексации выполняются извлечение содержимого, преобразование его в векторные представления и запись эмбедингов во векторное хранилище. Для текстовых документов применяется извлечение текста и его разбиение на фрагменты, для изображений формируются визуальные представления и, при необходимости, извлекается текст методом OCR, для видео используется выборка кадров и

построение агрегированного embedding-представления. Поддерживаемые типы данных и способ их обработки приведены в таблице 3.2.2.

Таблица 3.2.2 — Поддерживаемые типы данных и способ их обработки

Тип данных	Форматы	Способ обработки	Результат индексации
Текстовые файлы	txt, md	Чтение текста и разбиение на чанки	Text embeddings
Документы	pdf, docx	Извлечение текстового содержимого и чанкинг	Text embeddings
Изображения	jpg, jpeg, png, webp	Формирование image embedding, OCR	Image и text embeddings
Видео	mp4, mov, avi, mkv	Выборка кадров и построение агрегированного video embedding	Image и text embeddings

Мультимодальная обработка данных реализована через единый контур embeddings и retrieval. Для разных типов данных используются отдельные векторные пространства и соответствующие коллекции хранилища. Это позволяет выполнять поиск по текстовым документам, изображениям и видео, а также поддерживать смешанные сценарии, в которых пользовательский запрос может быть текстовым или сопровождаться графическим вложением. При наличии изображения-запроса система использует его embedding для поиска по визуальной коллекции, а при текстовом запросе выполняет поиск по нескольким модальностям с последующим объединением результатов.

RAG-пайплайн включает этапы извлечения релевантного контекста и генерации ответа. После поступления запроса система строит его векторное представление, выполняет поиск в текстовой, визуальной и видео-коллекциях, объединяет найденные результаты и формирует контекст,

который передаётся в LLM-модуль. Генеративная модель не работает изолированно, а использует найденные фрагменты как внешний источник знаний, что снижает вероятность нерелевантных и недостоверных ответов. В ответ пользователю возвращаются текст результата и сведения об использованных источниках.

Архитектура LLM-модуля предусматривает возможность подключения нескольких backend-реализаций текстовых и мультимодальных моделей, что обеспечивает гибкость выбора генеративного ядра. Конфигурация системы позволяет задавать используемую модель, ограничивать объём контекста и адаптировать параметры генерации под вычислительную среду. Для хранения истории запросов используется реляционная база данных, а механизмы разграничения доступа обеспечивают работу только с данными конкретного пользователя. Для разделения пользовательских корпусов документов в retrieval-контуре применяется фильтрация по идентификатору владельца, что исключает смешивание данных разных пользователей.

Таким образом, реализация веб-приложения мультимодального поиска выполнена в виде распределённой многокомпонентной системы, обеспечивающей полный цикл работы с данными: от загрузки и хранения файлов до их асинхронной индексации, семантического поиска и генерации ответа пользователю. Программная реализация отражает ранее спроектированную архитектуру и создаёт основу для практического применения RAG-подхода в задачах поиска и анализа мультимодальной информации.

3.3. Тестирование и анализ результатов работы системы

Тестирование разработанного веб-приложения мультимодального поиска проводилось с целью проверки корректности работы основных модулей системы, устойчивости прикладной логики и согласованности взаимодействия между серверной частью, подсистемой RAG, хранилищами данных и инфраструктурными компонентами. Проверка охватывала как пользовательские сценарии работы, так и внутренние сервисные механизмы, связанные с обработкой запросов, формированием эмбеддингов, поиском релевантного контекста, подготовкой prompt для LLM и экспортом эксплуатационных метрик.

Тестирование в проекте реализовано на основе pytest и включает модульные и интеграционные проверки. Тесты разделены на две основные группы, содержащую модульные проверки отдельных функций и сервисных сценариев, интеграционные тесты, выполняемые через FastAPI TestClient. Такой подход позволяет одновременно контролировать как внутреннюю корректность отдельных компонентов, так и работоспособность системы на реальных сценариях.

Модульные тесты используются для проверки наиболее критичных элементов прикладной логики. В проекте проверяется валидация входных моделей, обработка конфигурации, соблюдение лимитов и квот, маршрутизация запросов к провайдерам эмбеддингов, корректность мультимодального поиска и формирование запросов к модели. Проведённые проверки подтвердили, что система отклоняет некорректный пользовательский ввод, включая пустые текстовые запросы и неверные пути к мультимодальным вложениям. Отдельно проверена корректность чтения конфигурации, соблюдение ограничений по использованию ресурсов и работа механизмов защиты от неконтролируемой нагрузки.

Существенное внимание уделялось тестированию логики мультимодального поиска. Проверялась корректность обращения к нужным

типам векторных коллекций, формирование поиска на основе изображения при наличии графического вложения, а также исключение исходных файлов из результатов поиска при необходимости. Кроме того, были протестированы сценарии, связанные с подготовкой запросов к генеративной модели, включая передачу визуального контекста и формирование итогового промпта. Это подтверждает, что мультимодальный контур системы реализован не формально, а действительно учитывает различия между текстовыми и визуальными сценариями обработки запроса.

Интеграционное тестирование ориентировано на проверку согласованной работы прикладного API и инфраструктурных компонентов. В качестве показательного сценария проверяется ввод текста с заготовленным промптом: тест выполняет HTTP-запрос к приложению, убеждается в корректности ответа и дополнительно подтверждает, что после успешного вызова увеличивается счётчик запросов в метриках приложения. Такой сценарий важен тем, что проверяет не только сам прикладной эндпоинт, но и его связь с подсистемой мониторинга, то есть демонстрирует корректную работу сразу нескольких компонентов системы в едином контуре.

Для анализа работоспособности системы в процессе тестирования использовалась подсистема мониторинга на базе Prometheus и Grafana. Веб-бэкенд и сервисы обработки экспортируют метрики, отражающие состояние основных контуров приложения. В составе мониторинга фиксируются HTTP-метрики, показатели работы RAG-пайплайна, метрики формирования эмбедингов, а также показатели, связанные с индексацией и обработкой заданий. К числу контролируемых характеристик относятся количество запросов, длительность обработки, число найденных документов, глубина очереди задач, длительность индексации и сквозная задержка обработки. Наличие такой системы мониторинга позволяет оценивать не только факт успешного выполнения запросов, но и поведение приложения

под нагрузкой, стабильность фоновой обработки и согласованность работы отдельных подсистем.

Таблица 3.3.1 — Поддерживаемые группы метрик

Группа метрик	Назначение
HTTP	Контроль количества запросов, ошибок и задержек API
RAG	Мониторинг числа запросов, длительности обработки и количества найденных документов
Embeddings	Контроль частоты и длительности формирования векторных представлений
Ingest	Мониторинг очереди обработки, времени индексации, повторных попыток и DLQ

Prometheus в рамках разработанного решения используется как средство сбора и агрегации метрик, а Grafana — как средство визуального анализа состояния системы. Для проекта подготовлены панели мониторинга, отражающие частоту HTTP-запросов, уровень ошибок, задержки обработки, интенсивность работы RAG-контура, характеристики подсистемы embeddings, а также общие показатели использования ресурсов серверной части. Использование визуальных дашбордов позволяет оценивать поведение системы в процессе выполнения тестовых сценариев, выявлять узкие места и получать наглядное представление о стабильности работы приложения. Таким образом, Prometheus и Grafana выступают не только средствами эксплуатационного мониторинга, но и инструментами практической проверки работоспособности программного продукта.

Автоматизация тестирования встроена в контур непрерывной интеграции на базе GitHub Actions. При внесении изменений в основную ветвь проекта автоматически выполняется запуск CI-пайплайна, включающего подготовку окружения, установку зависимостей и выполнение набора автоматизированных тестов. Это позволяет регулярно проверять

стабильность программного решения, своевременно выявлять регрессии и поддерживать единый стандарт качества разработки. Автоматический запуск тестов в CI подтверждает, что проверки являются частью реального процесса разработки и сопровождения проекта, а не выполняются эпизодически.

По результатам проведённого тестирования установлено, что разработанная система корректно выполняет основные функции, предусмотренные архитектурой мультимодального поиска и анализа данных. Подтверждена работоспособность механизмов загрузки и обработки данных, корректность поиска и генерации ответов, устойчивость прикладной логики к типовым ошибкам ввода, а также возможность наблюдения и анализа состояния системы с помощью эксплуатационных метрик. Таким образом, разработанное веб-приложение является работоспособным в рамках проверенных сценариев и соответствует поставленным задачам по реализации мультимодального поиска на основе RAG-архитектуры и LLM-модуля.

Заключение

Проведённое исследование достигло своей цели — разработано веб-приложение мультимодального поиска и анализа данных на основе RAG-архитектуры и LLM-модуля, обеспечивающее обработку разнородных данных, их индексацию, семантический поиск и генерацию естественно-языковых ответов с использованием внешнего контекста. В ходе выполнения работы были исследованы методы мультимодального поиска, подходы к построению Retrieval-Augmented Generation-систем, принципы формирования векторных представлений и механизмы интеграции генеративных моделей с внешними источниками данных. Результаты выполнения поставленных задач подтверждают достижение цели работы.

В ходе анализа предметной области выявлены ограничения традиционных полнотекстовых методов поиска, что позволило обосновать использование векторных представлений, специализированных хранилищ эмбедингов и архитектуры Retrieval-Augmented Generation.

При исследовании подходов к реализации мультимодального поиска разработана модель обработки данных, включающая индексацию файлов, извлечение содержимого, построение эмбедингов, поиск релевантных фрагментов и генерацию ответа в рамках единого конвейера обработки запроса.

В процессе проектирования сформированы требования к системе, определены пользовательские сценарии, разработаны архитектурная модель веб-приложения и структура данных. Предложенное решение включает клиентский интерфейс, серверный API-модуль, модуль фоновой обработки и контуры файлового, реляционного и векторного хранения данных.

В практической части реализовано веб-приложение мультимодального поиска и анализа данных, поддерживающее загрузку файлов, асинхронную

индексацию, формирование векторных представлений, мультимодальный поиск и генерацию ответов на основе Retrieval-Augmented Generation.

Оценка эффективности показала работоспособность основных модулей системы и корректность ключевых пользовательских сценариев. Использование автоматизированного тестирования и CI-конвейера обеспечило воспроизводимость проверок и повысило надёжность реализации.

Дополнительно рассмотрены инфраструктурные и вспомогательные технологии, обеспечивающие развертывание, сопровождение и дальнейшее развитие программного решения.

В заключение, в ходе исследования была поставлена цель — разработка веб-приложения поиска и анализа мультимодальных данных на основе архитектуры генерации с дополненной выборкой и модулей генеративных языковых моделей. Для её достижения были выполнены следующие задачи:

1. Проведён анализ предметной области мультимодального поиска и анализа данных, выявлена проблема и сформулированы требования к решению;
2. Разработана интеллектуальная модель мультимодального поиска и анализа данных на основе больших языковых моделей и RAG-архитектуры;
3. Спроектировано и реализовано веб-приложение в виде информационной системы для поиска и анализа мультимодальных данных с интеграцией визуально-языковой модели;
4. Проведена оценка эффективности разработанного решения на тестовых примерах и выполнен анализ полученных результатов.

Список использованных источников

- [1] Большие языковые модели (LLM) // Yandex Cloud: Глоссарий. URL: <https://yandex.cloud/ru/docs/glossary/llm> (дата обращения: 08.10.2025).
- [2] Визуально-языковые модели (VLM) // Yandex Cloud: Блог. URL: <https://yandex.cloud/ru/blog/vlm-visual-language-models> (дата обращения: 08.10.2025).
- [3] RAG-системы: что это такое, принципы работы, архитектура и ограничения // Хабр URL: <https://habr.com/ru/articles/989000/> (дата обращения: 08.10.2025).
- [4] Idan O., Vexler V., Lederman G., Sivov D., Cohen Zada A. Few Shots Text to Image Retrieval: New Benchmarking Dataset and Optimization Methods // arXiv:2404.15790v1 [cs.CV]. - 2026. - С. 1-8.
- [5] Томилов Н.А., Туров В.П., Бабаянц А.А., Платонов А.В. Метод хранения векторных представлений в сжатом виде с применением кластеризации // Научно-технический вестник информационных технологий, механики и оптики. - 2024. - №1. - С. 1-5.
- [6] Березовский Б.Ю. Методы и технологии повышения эффективности работы организации на основе искусственного интеллекта // Международный научный журнал «Вестник науки». - 2025. - №6 (87), том 3. - С. 1-5.
- [7] Barbany O., Huang M., Zhu X., Dhua A. Leveraging Large Language Models for Multimodal Search // arXiv:2404.15790v1 [cs.CV]. - 2024. - С. 1-7.
- [8] Шмат А.В. Применение больших языковых моделей и технологии Retrieval-Augmented Generation для корпоративных ассистентов // Известия ТулГУ. Технические науки. - 2024. - №10. - С. 2-5.

- [9] Функциональные и нефункциональные требования: различия и особенности // Яндекс практикум URL: <https://practicum.yandex.ru/blog/funkcionalnye-i-nefunkcionalnye-trebovaniya/> (дата обращения: 10.10.2025).
- [10] Архитектура ИТ решений. Часть 1. Понятие «Архитектура» // Хабр URL: <https://habr.com/ru/articles/955218/> (дата обращения: 24.10.2025).
- [11] Python 3 Documentation // Python.org. URL: <https://docs.python.org/3/> (дата обращения: 26.10.2025).
- [12] FastAPI Documentation // FastAPI. URL: <https://fastapi.tiangolo.com/> (дата обращения: 26.10.2025).
- [13] REST API // Yandex Cloud: Глоссарий. URL: <https://yandex.cloud/ru/docs/glossary/rest-api> (дата обращения: 26.10.2025).
- [14] Материал о REST API // Habr: Песочница. URL: <https://habr.com/ru/sandbox/171066/> (дата обращения: 26.10.2025).
- [15] Flask Documentation // Flask. URL: <https://flask.palletsprojects.com/en/stable/> (дата обращения: 26.10.2025).
- [16] Django REST framework — API guide // Django REST framework. URL: <https://www.django-rest-framework.org/> (дата обращения: 26.10.2025).
- [17] Статья о FastAPI и асинхронных сервисах // Habr. URL: <https://habr.com/ru/articles/776538/> (дата обращения: 26.10.2025).
- [18] LangChain Documentation // LangChain. URL: <https://docs.langchain.com/> (дата обращения: 26.10.2025).

- [19] API Reference // OpenAI API. URL: <https://platform.openai.com/docs/api-reference/introduction> (дата обращения: 10.11.2025).
- [20] Hugging Face — платформа для машинного обучения // Hugging Face. URL: <https://huggingface.co/> (дата обращения: 10.11.2025).
- [21] What is LangChain? // AWS. URL: <https://aws.amazon.com/ru/what-is/langchain/> (дата обращения: 10.11.2025).
- [22] Qdrant Documentation // Qdrant. URL: <https://qdrant.tech/documentation/> (дата обращения: 10.11.2025).
- [23] pgvector Vector Store Integration // LangChain. URL: <https://docs.langchain.com/oss/python/integrations/vectorstores/pgvector> (дата обращения: 10.11.2025).
- [24] What is k-Nearest Neighbors (kNN)? // IBM Think. URL: <https://www.ibm.com/think/topics/knn> (дата обращения: 11.11.2025).
- [25] PostgreSQL Official Site // PostgreSQL. URL: <https://www.postgresql.org/> (дата обращения: 11.11.2025).
- [26] gRPC — A high performance, open source universal RPC framework // gRPC. URL: <https://grpc.io/> (дата обращения: 11.11.2025).
- [27] What is Structured Query Language (SQL)? // IBM Think. URL: <https://www.ibm.com/think/topics/structured-query-language> (дата обращения: 11.11.2025).
- [28] Docker Documentation // Docker. URL: <https://docs.docker.com/> (дата обращения: 13.11.2025).

- [29] Kubernetes Documentation // Kubernetes. URL: <https://kubernetes.io/docs/home/> (дата обращения: 13.11.2025).
- [30] TypeScript Documentation // TypeScript. URL: <https://www.typescriptlang.org/docs/> (дата обращения: 14.11.2025).
- [31] JavaScript — MDN Web Documentation // Mozilla Developer Network. URL: <https://developer.mozilla.org/en-US/docs/Web/JavaScript> (дата обращения: 14.11.2025).
- [32] Nuxt 4 Documentation. Getting Started // Nuxt. URL: <https://nuxt.com/docs/4.x/getting-started/introduction> (дата обращения: 14.11.2025).
- [33] Vue.js Guide. Introduction // Vue.js. URL: <https://vuejs.org/guide/introduction> (дата обращения: 14.11.2025).
- [34] Статья о Nuxt и SSR-приложениях // Habr. URL: <https://habr.com/ru/articles/798585/> (дата обращения: 19.11.2025).
- [35] Статья о Vue 3 и композиционном API // Habr. URL: <https://habr.com/ru/articles/568280/> (дата обращения: 19.11.2025).
- [36] Install Tailwind CSS with Vite // Tailwind CSS. URL: <https://tailwindcss.com/docs/installation/using-vite> (дата обращения: 19.11.2025).
- [37] Vite Documentation // Vite. URL: <https://vite.dev/> (дата обращения: 19.11.2025).
- [38] Webpack Documentation // Webpack. URL: <https://webpack.js.org/> (дата обращения: 20.11.2025).
- [39] ESLint Documentation // ESLint. URL: <https://eslint.org/docs/latest/> (дата обращения: 20.11.2025).

- [40] Prettier Documentation // Prettier. URL: <https://prettier.io/docs/> (дата обращения: 20.11.2025).
- [41] UV Documentation // Astral. URL: <https://docs.astral.sh/uv/> (дата обращения: 20.11.2025).
- [42] pip User Guide // Python Package Index. URL: <https://pip.pypa.io/en/stable/> (дата обращения: 20.11.2025).
- [43] Poetry Documentation // Python Poetry. URL: <https://python-poetry.org/docs/> (дата обращения: 20.11.2025).
- [44] Prometheus Documentation. Introduction // Prometheus. URL: <https://prometheus.io/docs/introduction/overview/> (дата обращения: 29.11.2025).
- [45] Grafana Documentation // Grafana Labs. URL: <https://grafana.com/docs/> (дата обращения: 29.11.2025).
- [46] pre-commit — A framework for managing and maintaining multi-language pre-commit hooks // pre-commit. URL: <https://pre-commit.com/> (дата обращения: 29.11.2025).
- [47] Material for MkDocs Documentation // MkDocs-Material. URL: <https://squidfunk.github.io/mkdocs-material/> (дата обращения: 29.11.2025).
- [48] Markdown Guide // Markdown Guide. URL: <https://www.markdownguide.org/> (дата обращения: 29.11.2025).
- [49] Mermaid — Diagramming and charting tool // Mermaid. URL: <https://mermaid.js.org/> (дата обращения: 04.12.2025).
- [50] Conda — Conda Documentation // Conda. URL: <https://docs.conda.io/en/latest/> (дата обращения: 20.11.2025).

[51] PromQL — Prometheus Documentation. Querying basics // Prometheus.
URL: <https://prometheus.io/docs/prometheus/latest/querying/basics/> (дата
обращения: 29.11.2025).

Приложения

Таблица 3.1.1 — Сравнение Qdrant и pgvector по ключевым характеристикам

Критерий	Qdrant	pgvector	Вывод
Алгоритм поиска	HNSW	k-NN [24]	Qdrant быстрее при больших коллекциях
Масштабируемость	Кластеры, шардинг	Только через PostgreSQL [25]	Qdrant подходит для системы с ростом данных
API	REST, gRPC [26]	SQL [27]	Qdrant проще интегрировать с LangChain
Формат хранения	Vector + payload	Table rows	Qdrant поддерживает гибридный поиск «вектор + метаданные»
Производительность	10^6 – 10^8 объектов	10^4 – 10^5 объектов	Qdrant эффективен для мультимодальных систем

Таблица 3.1.2 — Характеристика технологий серверной части

Технология	Назначение	Ключевые преимущества
Python	Реализация серверной логики, взаимодействие с моделями и обработка данных	Развитая экосистема библиотек для работы с эмбедингами, поддержка асинхронных фреймворков, низкий порог интеграции с ML-инструментами
FastAPI	Построение REST и WebSocket API	Асинхронная обработка запросов, автоматическая генерация OpenAPI-спецификации,

		высокая производительность при параллельной нагрузке
LangChain	Формирование RAG-цепочек и взаимодействие с генеративными моделями	Поддержка поиска, постобработки, унификация доступа к внешним LLM и VLM, modular pipeline
Qdrant	Хранение и поиск векторных представлений	Алгоритм HNSW для быстрого поиска ближайших соседей, гибридная фильтрация по метаданным, поддержка кластеризации
Supabase	Хранение операционных и пользовательских данных, управление аутентификацией и авторизацией, поддержка прикладной базы знаний	Основан на PostgreSQL, поддерживает готовые механизмы аутентификации и управления доступом, упрощает разработку серверной части

Таблица 3.1.3 — Технологии контейнеризации и оркестрации

Технология	Назначение	Ключевые характеристики
Docker	Контейнеризация компонентов системы	Изоляция окружений, воспроизводимость, независимость зависимостей
Kubernetes	Управление кластером контейнеров	Автоматическое масштабирование, балансировка нагрузки, мониторинг состояния

Таблица 3.1.4 — Характеристика технологий клиентской части

Технология	Назначение	Преимущества
TypeScript	Типизация кода	Надёжность, снижение ошибок
Nuxt 4	Клиентский интерфейс	SSR, высокая производительность

Tailwind CSS	Стилизация компонентов	Быстрое прототипирование
Vite	Сборка проекта	Высокая скорость, простота настройки

Таблица 3.1.5 — Сравнение технологий управления пакетами

Инструмент	pip	UV	Poetry	Conda
Управление окружением	Нет	Да	Да	Да
Разрешение зависимостей	Нет	Да	Да	Да
Lock Files	Нет	Да	Да	Да
Управление версией Python	Нет	Да	Нет	Да
Сборка и публикация пакетов	Нет	Да	Да	Нет
Скорость	Медленно	Очень быстро	Быстро	Медленно
Поддержка non-python зависимостей	Нет	Нет	Нет	Да

Таблица 3.1.6 — Характеристика дополнительных технологий

Технология	Назначение	Ключевые характеристики
UV	Управление зависимостями и пакетами для среды разработки	Высокая скорость установки библиотек, оптимизация сборки Python-проектов, наличие кроссплатформенной поддержки
Prometheus	Сбор и хранение метрик состояния системы	Модель временных рядов, язык запросов PromQL [51], поддержка экспортеров и мониторинга микросервисов

Grafana	Визуализация метрик и построение аналитических панелей	Поддержка подключаемых источников данных, гибкие дашборды, инструменты аналитики состояния системы
pre-commit	Контроль качества исходного кода	Автоматическая проверка формата, статический анализ, выполнение тестов перед фиксацией изменений
mkdocs	Автоматизация подготовки документации	Генерация статических сайтов, структурирование технических материалов, поддержка разметки Markdown

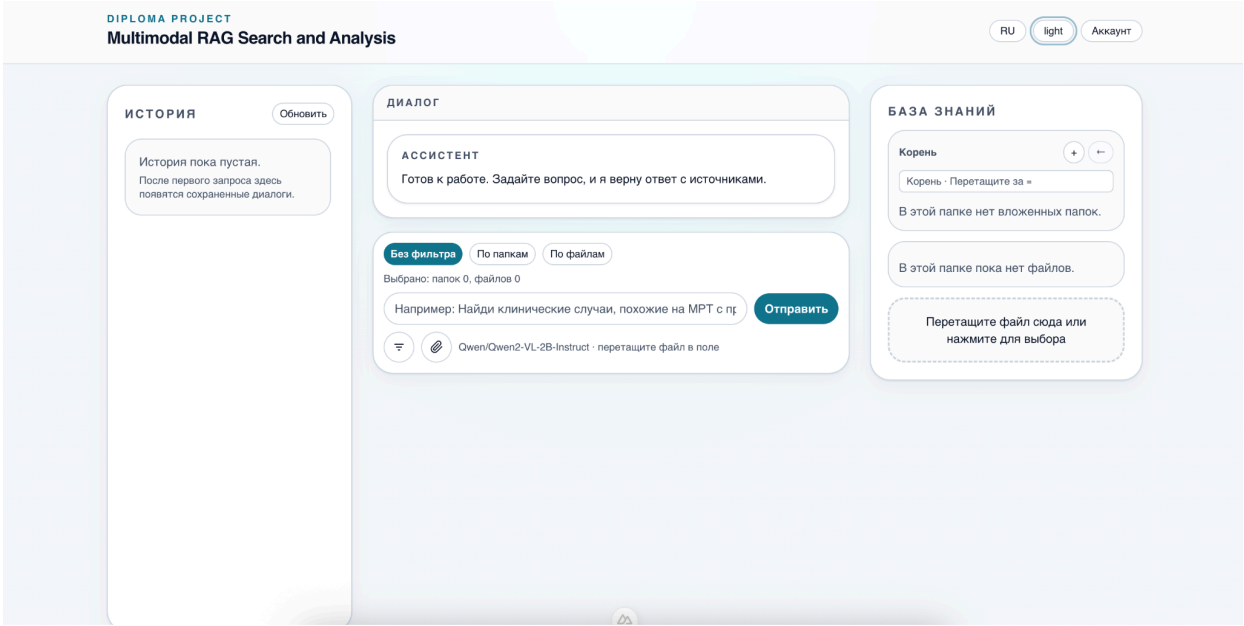


Рисунок 3.2.2 — Интерфейс системы

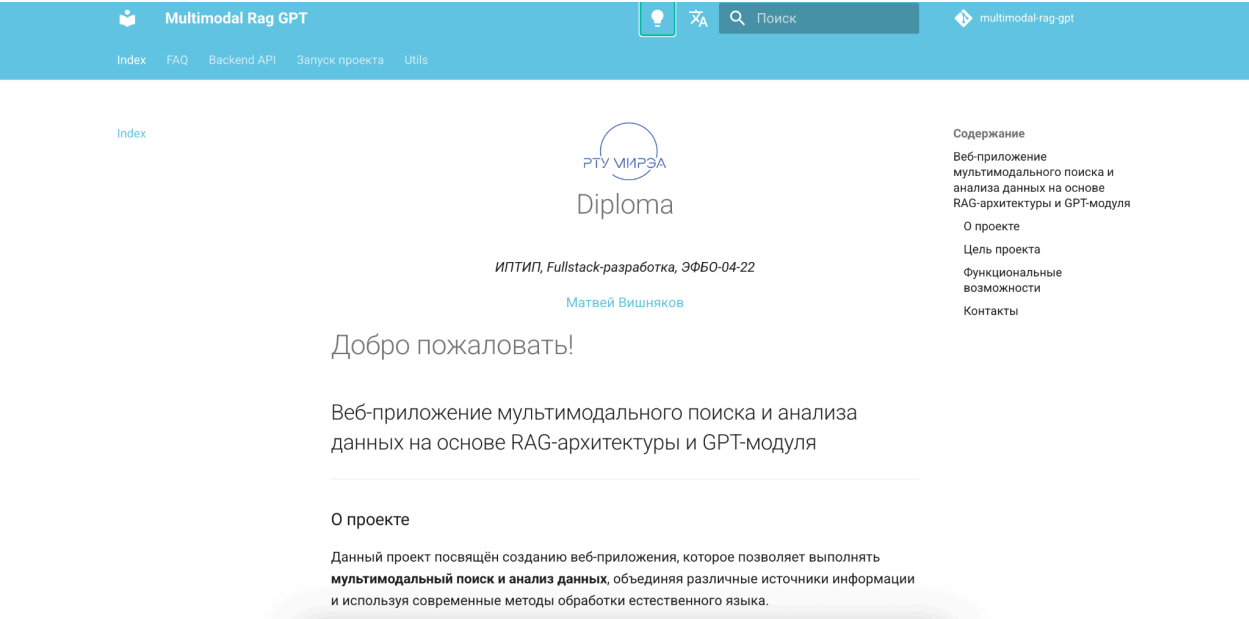


Рисунок 3.2.3 — Пользовательская документация

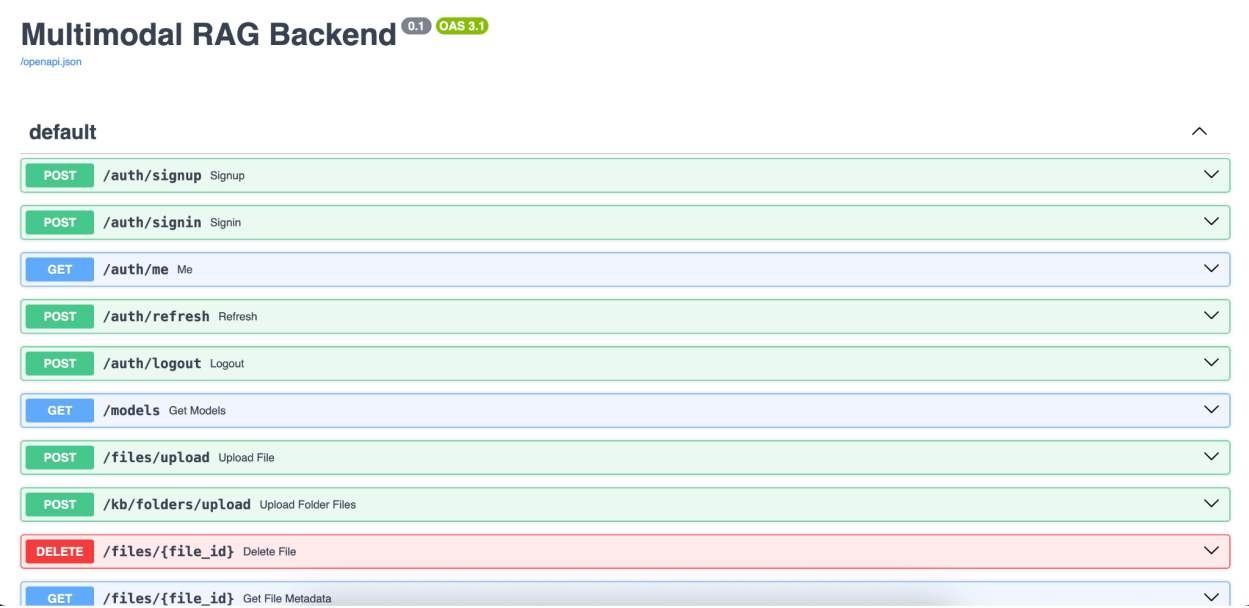


Рисунок 3.2.4 — API-эндпоинты